

01

바이브코딩이란?
그리고 프롬프트

오늘의 목표

"프롬프트 잘 쓰기"를 넘어서, AI 개발의 전체 흐름을 이해합니다.

1

바이브코딩 이해하기

기존 개발 방식과
무엇이 같고
무엇이 다른지
설명할 수 있다.

2

프롬프트 구조화

명확성·구체성·
맥락·제약을 넣어
프롬프트 작성하기

3

4

Claude Artifact

Claude Artifacts
도구 익히기

바이브코딩, 무엇이 달라졌을까?

코드를 작성하는 기술에서 · 의도를 설계하고 검증하는 기술로

바이브코딩(Vibe Coding)이란?

자연어로 의도를 전달하고 AI가 생성한 결과를 반복적으로 실행·검증·수정하는 개발 방식

"코드를 직접 타이핑하는 시대에서,
자연어로 의도를 전달하고 결과를 실행·검증하는 시대"

- 2025년 2월, 전 OpenAI 수석연구원 안드레이 카파시(Andrej Karpathy)가 제안한 새로운 개발 방식
- AI와 대화를 주고받으며 프로그램을 만들어 가는 개발 방식
- 단, 말만 하면 되는 마법이 아닙니다 무엇을 말하느냐가 결과를 결정합니다

바이브코딩(Vibe Coding)이란?

자연어로 의도를 전달하고 AI가 생성한 결과를 반복적으로 실행·검증·수정하는 개발 방식

전통적 개발

문법 학습
직접 코드 작성
디버깅, 수정

AI 보조 개발

코드 일부 생성
사람이 통합·수정
AI는 보조 역할

Vibe

자연어로 목표·제약 전달
AI가 구현
사람이 실행·검증·방향 결정

코딩을 전혀 안 하는 것이 아니라, **'코드 작성 비중'**이 줄고 **'기획·검증·설계'**의 비중이 급격히 커진 방식

왜 바이브 코딩에서도 '개발 지식 ' 이 필요할까?

AI가 만드는 코드의 위험

- **환각 (Hallucination)**
  존재하지 않는 라이브러리나 함수를 정당한 것처럼 사용
- **맥락 누실 (Context Drift)**
  대화가 길어지면 이전 요구사항을 잊거나 코드를 왜곡
- **보안 및 성능 방치**
  하드코딩된 API 키, 무한 루프, 비효율적 알고리즘 생성

사람이 갖춰야 할 핵심 역량

- **코드 리뷰 능력 (Code Literacy)**
  AI가 짜온 코드가 안전하고 맞는지 읽어내는 능력
- **문제 분할 (Decomposition)**
  거대한 앱을 작은 단위(기능별)로 쪼개어 지시하는 능력
- **디버깅 가이드 (Troubleshooting)**
  에러 메시지를 읽고 AI에게 정교한 힌트를 제공하는 능력

AI 코딩 도구 3가지 유형

어떤 모델을 쓰는지보다, AI가 코드베이스에 어디까지 접근하고 실행할 수 있는지가 중요합니다.

1 Chat형

ChatGPT, Claude, Gemini

- 대화창에
  코드나 에러 복사-붙여넣기

- **장점:**
  부분 문제 해결 및 학습에 탁월

- **단점:**
  전체 프로젝트 맥락유지 어려움

2 IDE 통합형

Cursor, Windsurf, VS Code Extension

- 코드 에디터 안에서
  AI가 프로젝트 전체를 인지

- **장점:** 자동 완성, 파일 간 참조, 개발 편의성

- **단점:** 도구별 사용법 숙지 필요

3 Agent형

Claude Code, Gemini CLI

- 터미널에서 로컬 폴더 읽기, 파일 생성, 테스트 실행까지 자율 수행

- **장점:**

프로젝트 단위의 복잡한 작업 자동화

- **단점:**
  권한 관리 및 변경 사항 검증 필수

AI는 어떻게 “개발 파트너”가 되었을까?

기술의 발전보다 AI에게 맡길 수 있는 작업 범위가 넓어졌다는 점입니다.

1 → 2 → 3 → 4

검색

사람이 답을 찾음

AI 챗봇

질문하면 코드 생성

AI Assistant

코드 맥락을 보고 제안

AI Agent

프로젝트를 읽고 작업 수행

Assistant는 “답을 제안”하고, Agent는 “목표를 향해 여러 단계를 수행”합니다.

프롬프트에서 “개발 명세”로

좋은 결과는 문장 기술보다 · 요구사항의 구조와 검증 기준에서 나옵니다.

프롬프트의 품질 = 결과를 판단할 수 있는 정보의 품질

단순히 길게 쓰는 것이 아니라, AI가 “무엇을 / 어떤 조건으로 / 어디까지” 해야 하는지 알려야 합니다.

❌ 모호한 프롬프트 (Bad)

학교 시간표 웹 페이지 예쁘게 만들어줘

디자인은 요즘 스타일로 해줘

학생들이 사용하기 편리하게해줘

❌ 문제점

'예쁘게', '요즘 스타일', '편했으면’ 등 주관적 단어로 가득함.
AI가 마음대로 추측해서 불필요한 기능 생성.

✅ 검증 가능한 명세 (Good)

고등학교 2학년 학생이 등교 직후 스마트폰(390px)으로 볼 1페이지 시간표 웹 앱을 작성해줘

요일별(월~금) 1~7교시 과목표시

오늘, 요일 카드 자동 강조

HTML, CSS, JA 한 파일로 작성해줘

지금은 기능이 없어 UI 레이아웃 뼈대만 작성해줘

✅ 장점:

타겟(학생), 화면크기(390px), 필수 기능,
제약조건(한 파일, 뼈대만)이 명확하여 정밀한 결과 출력.

프롬프트의 품질 = 결과를 판단할 수 있는 정보의 품질

단순히 길게 쓰는 것이 아니라, AI가 “무엇을 / 어떤 조건으로 / 어디까지” 해야 하는지 알려야 합니다.

✕ 모호한 프롬프트 (Bad)

- "편의점 가서 뭐 좀 사와"

문제점
- 사용자와 목적이 없음
- 핵심 기능의 우선순위가 없음
- 성공 여부를 판단할 기준이 없음

✓ 검증 가능한 명세 (Good)

- "학교 앞 편의점에서
  삼각김밥 참치마요 2개"
- "2천 원 안에서, 없으면
  전주비빔 맛으로"

핵심: “좋아 보이게”보다 “성공 조건을 검증할 수 있게”

좋은 프롬프트의 네 가지 조건

1 **명확성** 주관적이거나 모호한 단어 제거

❌ "글자를 좀 예쁘고 크게 해줘"
✅ "제목 글자는 24px Bold, 본문은 16px Regular 적용"

2 **구체성** 필요한 요소와 동작 순서를 항목화

❌ "투두리스트 만들어줘"
✅ "입력창 1개, 추가 버튼 1개, 삭제 아이콘, 체크박스 포함"
.

3 **맥락** 누가, 언제, 어떤 환경에서 쓰는지 전달

❌ "타이머 앱 짜줘"
✅ "시험기간에 고등학생이 사용할 50분 스톱워치"

4 **제약** AI가 하지 말아야 할 경계를 설정

❌ "수정해줘"
✅ "기존 JS 로직은 유지하고, CSS 스타일만 변경해줘"

5W1H는 좋은 “아이디어 정리 시작점”

1 Why 왜 만드는가

해결하려는 불편함과 목적

교실에서 아침마다 시간표 확인하느라 시간을 허비함

2 Who 누가 쓰는가

대상 사용자와 그 사람의 상황

고등학교 2학년 반 학생 전체

3 What 무엇을 만드는가

핵심 기능과 화면에 담길 내용

요일별 시간표 + 오늘의 준비물 알림 메모장

4 When 언제 쓰는가

사용 시점 등교 전, 시험 기간, 매일 밤

매일 등교 직후 8:30 AM

5 Where 어디서 쓰는가

사용 환경 휴대폰인가 노트북인가

모바일 브라우저
(주로 모바일 화면 크기)

6 How 어떻게 만드는가

구현 방식과 필요한 기술 수준

HTML/CSS/JS (Claude Artifacts 1개 파일로 완출)

프롬프트 예시 · “기능 요청”이 아니라 “검증 가능한 작업”으로

한 번에 완제품을 요구하지 않고, 현재 단계와 수정 범위를 명확하게 지정합니다.

학생용 프롬프트 예시

나는 고등학교 2학년이고, 아침에 쓸 시간표 확인 웹페이지를 만들고 싶어.

왜 : 매일 아침 교실에서 시간표를 확인하느라 시간을 쓴다
누가 : 나와 우리 반 학생들
무엇을 : 요일별 시간표, 오늘 수업 강조 표시, 준비물 메모
언제 : 등교 직후와 자기 전
어디서 : 주로 휴대폰 브라우저
어떻게 : HTML, CSS, JavaScript 한 페이지로

지금은 화면 구조만 만들어줘. 기능은 아직 구현하지 말고,
어떤 요소가 어디에 들어가는지 볼 수 있게만 해줘.

마지막 두 줄이 제약 조건입니다. 이 문장이 있고 없고에 따라 돌아오는 코드의 양이 다섯 배 차이 납니다.

시연–[잘못된 프롬프트 구출하기]

한 번에 완제품을 요구하지 않고, 현재 단계와 수정 범위를 명확하게 지정합니다.

퀴즈: 이 프롬프트의 문제는?

할 일 관리 앱 하나 만들어줘
디자인 깔끔하게, 오늘할일 등록하고 지울 수 있게

❓ **함께 생각해보기:**

1. 저장 방식이 정의되어 있는가? (새로고침하면?)
2. 화면 구성과 색상 스타일은 지정되었는가?
3. 한 번에 너무 많은 기능을 요청했는가?

시연 가이드

- **Step 1:** 위 모호한 프롬프트를 Claude에 그대로 입력 후 출력 결과 확인 (엉성한 UI)
- **Step 2:** 5W1H와 제약조건을 적용한 구조화 프롬프트 재입력
- **Step 3:** Claude Artifacts가 실시간으로 웹 앱을 그려내는 과정 비교

Claude

미국의 AI 기업 Anthropic(앤트로픽)이 만든 AI 도구

2021년 OpenAI Chat GPT 만든회사에서 나온 남매가 창업한 회사



OpenAI가 영리화의 길을 걷자

‘AI는 안전이 먼저‘ 라는 가치를 지키기 위해 설립

영리기업과 달리 공익 목적공익기업형태(PBC)

Claude Artifacts = 빠른 프로토타이핑 공간

대화와 결과물을 분리해, 웹앱·도구·시각화 등을 즉시 만들고 수정하며 확인할 수 있습니다.

대화 창(Left)과 실시간 실행 창(Right)이 분리된 샌드박스 환경.

- **왼쪽 (Chat):** 프롬프트를 주고받으며 피드백 수정
- **오른쪽 (Artifact/Preview):** Rendered HTML/React 앱을 즉시 테스트

✓ 복잡한 개발 환경 설정(Node.js, VS Code) 없이 브라우저에서 바로 동작

✓ 코드와 대화 내역이 섞이지 않아 버전 관리 및 비교가 수월함

Claude Artifacts = 빠른 프로토타이핑 공간

대화와 결과물을 분리해, 웹앱·도구·시각화 등을 즉시 만들고 수정하며 확인할 수 있습니다.

대화 영역

“이 화면을 먼저 뼈대로 만들어줘.”
“입력 데이터가 없을 때 상태를 추가해줘.”
“모바일 390px 기준으로 다시 설계해줘.”

→ 요구사항을 반복적으로 구체화

Artifact / Preview

실행 가능한 프로토타입

UI 구조 확인
상호작용 테스트
변경 전·후 비교
공유 가능한 결과물

RUN

한 번에 만들지 않는 4단계 반복 빌드업

STEP 1
뼈대 구축

화면 구조와 영역 나누기. 기능 없이 HTML/CSS 박스만 배치.

STEP 2
기능 붙이기

버튼 클릭, 데이터 입력, 삭제 등 core JavaScript 로직 연결.

STEP 3
디자인 정돈

색상, 폰트, 여백, 모바일 반응형 padding 정밀 조정.

STEP 4
점검 & 데이터

localStorage 데이터 저장 및 예외 상황 (예: 빈 입력) 테스트.

기획서를 통째로 주고 "이대로 만들어줘"라고 하면 코드가 한꺼번에 쏟아져 고칠 수 없게 됩니다.

MISSION

실습 : 할 일 관리 앱 개조하기

LEVEL 1

눈에 보이는 것 바꾸기

버튼 색상 변경 · 제목 문구 변경 기본 폰트 적용

LEVEL 2

기능 추가하기

삭제 버튼 추가 · 완료 항목 체크 박스 효과

LEVEL 3

데이터 다루기

검색 기능 · 완료/미완료 필터 버튼

LEVEL 4

기억하게 만들기

브라우저를 닫아도 목록이 남도록 저장 기능 추가

오늘의 핵심 요약

- **AI가 코드를 대신 만들어줄수록**
  사람은 더욱 정확하게 '요구사항을 정의하고 검증'해야 합니다.
- **프롬프트는 대화가 아니라 명세서**
  모호한 요청을 지우고 5W1H 및 제약조건을 명시하세요.
- **단계별 반복(Iterative) 빌드**
  뼈대 기능 디자인 검증의 4단계를 기억하세요.

오늘의 핵심

AI가 코드를 만들수록
사람은 더 정확하게 “정의하고 검증”해야 합니다.

VIBE CODING	자연어 → 생성 → 실행 → 검증 → 수정
PROMPT	문장이 아니라 작은 명세
PRD	범위와 성공 기준을 정하는 문서
ARTIFACTS	빠른 프로토타입과 UX 검증
CLAUDE CODE	코드베이스를 직접 다루는 Agent

NEXT · 문제 정의 → MVP → PRD.md 작성